University Institute of Engineering

Department of Computer Science & Engineering

Experiment: 8

Student Name: Jagrath **UID:24BDA70365**

Branch: CSE **Section/Group: AIT-KRG-GP2**

Semester: 4th **Date of Performance:31/03/2026**

Subject Name: DBMS

1. **Aim of the practical: T**o create and execute a stored procedure that retrieves employee data and calculates bonus dynamically using input parameters, demonstrating the use of functions and procedural logic in PostgreSQL.

2. Tool Used:

- **Database Management System:**
  - **PostgreSQL**
- **Database Administration Tool:**
  - **pgAdmin**

3. **Objective:**

To write and execute a PL/SQL stored procedure that dynamically accepts gender as an argument and computes the employee count corresponding to the given gender.

4. Practical / Experimental Steps

Step 1: Create the base table employees with attributes such as emp_id, emp_name, salary, and department.

Step 2: Insert sample records into the employees table to simulate real-world employee data.

Step 3: Define and create a stored procedure (function) GET_BONUS that accepts employee ID as an input parameter.

Step 4: Implement logic inside the procedure to retrieve the employee's salary and calculate the bonus using a percentage formula.

Step 5: Execute the stored procedure by passing dynamic input values using a DO block.

Step 6: Display the output using RAISE NOTICE to show employee ID and calculated bonus.

Step 7: Verify the correctness of the output and analyze how the procedure handles dynamic inputs and returns computed results.

5. I / O Analysis

A) Create Table

```
CREATE TABLE employees (
   emp_id INT  PRIMARY KEY,
   emp_name VARCHAR(100) NOT NULL,
   salary NUMERIC(10,2) NOT NULL,
   department VARCHAR(50)
);
```

```
CREATE TABLE

Query returned successfully in 138 msec.
```

B) Insert Sample Data

```
INSERT INTO employees (emp_id,emp_name, salary, department) VALUES
(1,'Alice', 50000, 'HR'),
(2,'Bob', 60000, 'IT'),
(3,'Charlie', 55000, 'Finance');
```

```
INSERT 0 3

Query returned successfully in 62 msec.
```

C) Functions and Their Output

```
CREATE OR REPLACE PROCEDURE get_Employee_Count_BY_Gender(IN P_GEN VARCHAR(20), OUT
COUNT_EMP INT, INOUT STATUS VARCHAR)
AS
        $$
                BEGIN

                        SELECT COUNT(*) INTO COUNT_EMP FROM EMPLOYEES WHERE GENDER=P_GEN;
                        STATUS:='SUCCESS';
    END;
        $$ LANGUAGE PLPGSQL


                DO
                $$
                DECLARE
                        GEN VARCHAR(20):='Male';
                        COUNT_BY_GEN INT;
                        STATUS VARCHAR(20):='Fail';
                BEGIN

                        CALL  get_Employee_Count_BY_Gender(GEN,COUNT_BY_GEN,STATUS);

                        RAISE NOTICE 'GENDER IS % YOUR COUNT IS %  AND STATUS IS
%',GEN,COUNT_BY_GEN,STATUS;

                END;

    $$
```

```
NOTICE:  GENDER IS Male YOUR COUNT IS 2  AND STATUS IS SUCCESS
DO

Query returned successfully in 68 msec.
```

```
CREATE OR REPLACE FUNCTION Get_Data()
RETURNS TABLE(name_emp VARCHAR(10), emp_salary NUMERIC(10,2))
AS
```

```
$$
BEGIN
RETURN QUERY SELECT emp_name, salary FROM employees;
END;
$$ LANGUAGE PLPGSQL;
```

```
CREATE FUNCTION

Query returned successfully in 60 msec.
```

```
SELECT * FROM Get_Data();
```

	name_emp character varying	emp_salary numeric
1	Amit	30000.00
2	Riya	35000.00
3	John	28000.00
4	Sneha	40000.00

6. Learning outcomes (What I have learnt):

- Understand the structure and components of a stored procedure.
- Implement IN and OUT parameters in PL/SQL procedures.
- Pass dynamic input values to stored procedures.
- Apply aggregate functions within procedures for data analysis.
- Develop reusable database logic for real-world business scenarios used in companies like CData and Toddle.